

KALAK 3PATTI

Live Tournament Handover Guide & Build Roadmap

Updated Player Availability & Seat Protection Rules (v2) · what exists today · how GitHub, Render and Upstash fit together · step-by-step plan to go live

Version	2 — 25 September 2026. Supersedes "Player Availability and Seat Protection Rules" (24 Sep 2026, prepared by Manus AI). All v1 rules are kept; v2 adds clarifications, the decisions taken since, and the roadmap.
Code repository	https://github.com/walling844-klk/kalak-3patti (branch main). State described here = commit 8877ae9 (25 Sep 2026).
Live site	https://kalak-3patti.onrender.com (Render web service, auto-named from the repo)
Storage	Upstash Redis (REST API) — keeps the tournament table alive across Render restarts.
Owner	A non-programmer game owner. Works in simple English, step by step, deploys by hand through the GitHub website and the Render dashboard. Times are Indian Standard Time (IST, UTC+05:30).
Written by	Claude (Anthropic), from the full project history, the repository, the Render logs and the live tests done up to 25 Sep 2026.

HOW TO USE THIS DOCUMENT (owner: show this file to any AI assistant)

Paste or attach this PDF and say: "Read this whole document. Open the GitHub repository it names and check that the code still matches section 2 and 4. Then continue with the first unfinished phase of the roadmap (section 9). Explain every step to me in simple English, give me ready-to-upload files with the exact GitHub path, test everything before you give it to me, and never ask me to paste passwords or tokens."

THREE THINGS THAT HAVE ALREADY GONE WRONG — DO NOT REPEAT

- 1) The game exists in **two versions** (a local file and the server-connected file). A local copy was once uploaded to GitHub by mistake and silently switched the live site back to browser-only storage. See section 4.
- 2) Render's free plan **wipes local files** on every restart, redeploy and after 15 idle minutes. Anything that must survive lives in Upstash, never in a file. See section 3.
- 3) The owner deploys by hand. A change is **not live** until it is committed to GitHub **and** Render's "Manual Deploy → Deploy latest commit" is pressed, and the deploy's Source commit is checked.

Contents

Contents	2
1. Ground rules for any AI working on this project	4
Decisions the owner has already made (DECIDED)	4
2. What exists today (VERIFIED 25 Sep 2026, commit 8877ae9)	4
3. Infrastructure and workflow: GitHub → Render, with Upstash	5
3.1 How the pieces fit	5
3.2 GitHub	5
3.3 Render (web service)	6
3.4 Upstash Redis	6
3.5 Deploy checklist (give this to the owner every time)	6
3.6 Local test set-up an AI should recreate	7
4. Repository map and the "two versions of the game" problem	7
5. The server today: API and data model (VERIFIED, server.js as on GitHub)	8
6. The game as implemented — the rulebook a server engine must reproduce	9
6.1 Hand ranking (highest first)	9
6.2 One hand, step by step	9
6.3 Actions	10
6.4 Sideshow	10
6.5 Boot escalation, stand up, game over	10
6.6 Timing constants in the newest game file	10
6.7 Bots (Computer seats)	11
6.8 Function index of the game page (to find things quickly)	11
7. What changed from the 24 September spec (v1 → v2)	11
8. Player Availability and Seat Protection Rules — v2 (final rule text)	12
8.1 Definitions	12
8.2 Player who never arrives	13
8.3 Player who disconnects or goes idle	13
8.4 The restricted bot (per hand)	13
8.5 Active-session seat protection	13
8.6 Returning during bot control, and device replacement	13
8.7 Automatic kick and chip distribution	14
8.8 Voluntary stand-up and admin kick	14
8.9 What the server must own, and how it survives a restart	14
8.10 Timing summary	14
9. Roadmap: from today to a live tournament	15
Phase 0 — Baseline and housekeeping (S)	15
Phase 1 — Server-side game engine (L)	15
Phase 2 — Real-time connection (M)	16

Phase 3 — Game page "online mode" (L)	16
Phase 4 — Match lifecycle (M)	17
Phase 5 — Availability and seat protection (L)	17
Phase 6 — Hardening (M)	17
Phase 7 — Rehearsal and launch (S)	17
Later (not part of the first live release)	17
10. Scenario test matrix (acceptance tests for Phases 2-5)	17
11. Traps to avoid and decisions still open	18
11.1 Traps (things that have misled or will mislead)	18
11.2 Open decisions (recommended default in bold; the owner may change any of them)	19
Appendix A — WebSocket protocol (DRAFT design, not implemented)	19
Appendix B — What one player's view of the table may contain	20
Appendix C — Glossary	20
Appendix D — Quick reference	20

1. Ground rules for any AI working on this project

- **Labels used in this document.** **VERIFIED** = checked in the repository, the Render logs or a live test on the date shown. **DECIDED** = a decision the owner made. **PROPOSED** = a recommended default the owner has not objected to but may change. **DRAFT** = a design idea that is not built. If something is not labelled, treat it as background.
- **Verify before you build.** Open the repository, read `server.js` completely and skim `public/KALAK_3PATTI.html` using the function index in section 6. If the code no longer matches this document, trust the code and tell the owner what changed.
- **The owner is not a developer.** Give exact clicks ("open the *public* folder → Add file → Upload files"). One question at a time. Say what is finished, what is not, and what could go wrong. Never assume the owner has a computer with Git installed.
- **Deliver tested, ready-to-upload files.** Run the server locally and test with several browsers acting as different phones (Playwright multi-context works well) before handing anything over. Give only the files that changed, with their exact repository path.
- **Secrets.** Real passwords and tokens must never appear in files, chats or this document. If the owner pastes a secret, tell them to reset it. Secrets live only in Render's Environment tab.
- **Do not shrink the game file.** The game page is about 11.6 MiB because five table pictures and sounds are embedded. The owner explicitly decided to keep it as it is.
- **Keep the offline game working.** "PLAY VS COMPUTER" runs entirely in the browser and must keep working exactly as it does today.

Decisions the owner has already made (DECIDED)

#	Decision	Where it matters
1	The tournament table must stay saved until the admin presses KILL THE TABLE (not lost on Render sleep or restart).	Upstash storage
2	KICK and KILL both need a confirmation tap ("YES, KICK" / "YES, KILL IT") to prevent mis-touches.	Admin panel
3	The game file stays un-shrunk (about 11.6 MiB).	Deployment
4	Admin password lives only in Render's environment (ADMIN_PASSWORD), never in the page.	Security
5	A second login with the same password is rejected while the first session is alive (never replaces it).	Spec section 8.5
6	Match end: no auto-exit. Show "MATCH HAS ENDED" with an EXIT THE TABLE button. Anyone joining the room after the match sees only the Congratulations message, the winner's name and "MATCH HAS ENDED — EXIT THE ROOM" until the admin kills the table.	Client + server
7	Absent-player rules exactly as in section 8 (10-minute bot window, 3 timed-out rounds, chip split, etc.).	Spec
8	Chip split applies only to automatic kicks (absentee / disconnecter). Voluntary stand-up keeps the existing surrender behaviour.	Spec section 8.7
9	Dates are typed and shown as DD/MM/YYYY; times are IST.	Admin panel
10	Table theme (5 pictures) is chosen per player, on that player's own device only.	Client

2. What exists today (VERIFIED 25 Sep 2026, commit 8877ae9)

Area	Status	Details
Offline game (PLAY VS COMPUTER)	Works	Complete Teen Patti engine, 8 seats, 7 bots with personalities, sideshow, boot escalation, match timer, recorded sounds, boot-collection and deal animations. All rules run inside the browser.

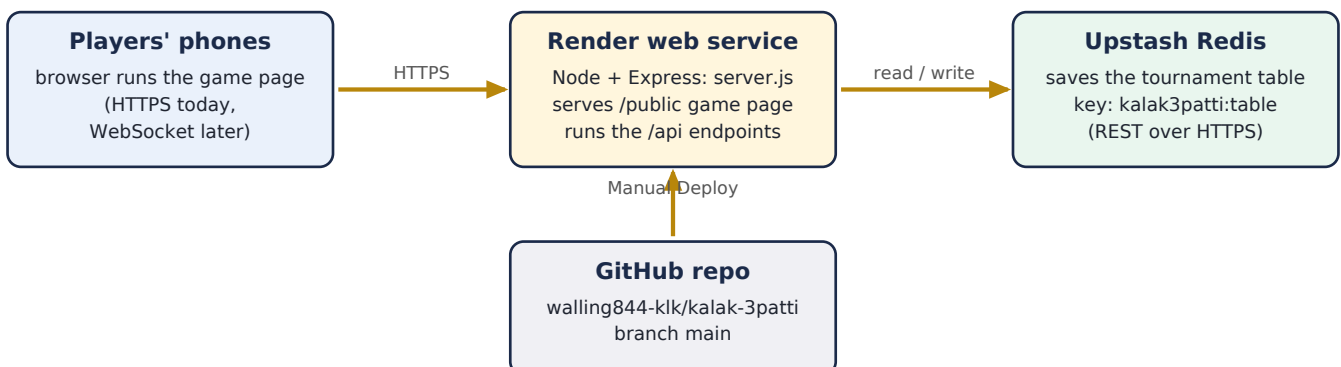
Area	Status	Details
Admin panel (server-connected)	Works	Login checked by the server. Create Table form (seats 2-8, names, passwords, Human/Computer per seat, observer password, chips, boot, blind, max blind/call, boot interval, start date DD/MM/YYYY + start time IST). Once created: "TABLE IS ONLINE", MATCH START editor (change date/time), KICK PLAYER list, KILL THE TABLE — the last two need a confirmation tap.
Table storage	Works	Saved on the server in Upstash Redis; survives Render sleep/restart/redeploy. Tested with a real restart.
Join Tournament	Works (holding screen only)	Player enters a password; the server finds the seat or the observer slot and answers with table settings only (never the password list). The "You're seated" screen shows a live countdown to the start time (IST), re-checks the server every 10 seconds, and closes with a message if the player is kicked or the table is killed. A kicked password is refused.
Match-ended support	Half built	The client can show the "MATCH HAS ENDED" screens and <code>/api/join</code> returns <code>ended:true</code> when the table has a result. Nothing writes a result yet, because no live match exists.
Live match (real players together)	NOT BUILT	Every phone runs its own private copy of the game. There is no server-side game engine, no real-time connection (WebSocket), no shared table, no automatic start, no results.
Player availability rules (section 8)	NOT BUILT	Needs the live match first: presence, timeout counting, restricted bot, 10-minute window, auto-kick, chip split.
Hidden settings	Stored, not used	<code>startingBlind</code> and <code>maxBlindCall</code> are saved with the table but no game rule uses them (the blind is always half the current stake and raises have no cap). The offline engine also ignores <code>chipsPerPlayer</code> , <code>startingBoot</code> and <code>bootIncreaseMinutes</code> (it uses constants 5000 / ₹10 / 5 minutes). The live engine must read them from the table config.

An earlier server-engine draft is NOT in the repository

During earlier chats a draft server game engine (`game.js`) was written but never finished, tested against the newest game file, or deployed. It is not on GitHub. Do not assume it exists. Build the engine fresh from section 6, which matches the current game.

3. Infrastructure and workflow: GitHub → Render, with Upstash

3.1 How the pieces fit



In words: the owner edits the code (with an AI) and uploads files to **GitHub**. In **Render** the owner presses *Manual Deploy* → *Deploy latest commit*; Render downloads the repo, runs `npm install`, then starts `node server.js`. The server serves the game page and answers `/api/...` calls. Anything that must survive a restart is written to **Upstash**.

3.2 GitHub

- Repository: <https://github.com/walling844-klk/kalak-3patti>, branch **main**, public.

- Files (VERIFIED): `server.js` (~16 KB), `package.json`, `package-lock.json`, `gitignore` (note: *no leading dot*, so it does nothing today), and `public/KALAK_3PATTI.html` (~11.6 MiB).
- The owner updates files with the GitHub website: open the folder → **Add file** → **Upload files** → drag the file (same name = replaces it) → **Commit changes**. GitHub's browser upload limit is 25 MiB per file, so the game page must stay below that.
- To check what is really on GitHub without logging in: the commit list is at <https://github.com/walling844-klk/kalak-3patti/commits/main.atom> and raw files at <https://raw.githubusercontent.com/walling844-klk/kalak-3patti/main/<path>>. (The GitHub REST API is rate-limited for anonymous callers.)

3.3 Render (web service)

Setting	Value (VERIFIED from the deploy logs, 24 Sep 2026)
Service URL	https://kalak-3patti.onrender.com
Runtime	Node.js 24.21.0 (Render default, no version pinned). Uses plain JavaScript (CommonJS, <code>require</code>).
Build command	<code>npm install</code>
Start command	<code>node server.js</code> (server listens on the PORT Render provides — 10000)
How deploys start	So far all deploys were manual ("Manually deployed by you via Dashboard"). Check whether Auto-Deploy is on; do not assume it is.
Plan	Free plan (assumed). Free web services spin down after about 15 minutes without inbound traffic; the first request afterwards can take up to a minute. The file system is ephemeral : files written at run time disappear on restart, redeploy and spin-down.

Environment variables (Render → the service → Environment). Names only — the values are secrets and must never be written down here:

Name	Purpose	If missing
ADMIN_PASSWORD	The admin panel password. Checked only on the server.	Server invents a random one at every start and prints it once in the log — so set it.
UPSTASH_REDIS_REST_URL	Address of the Upstash database (starts with <code>https://</code>).	Server falls back to a local file → the table is lost on restart.
UPSTASH_REDIS_REST_TOKEN	Secret token for that database.	Same as above.
PORT	Set by Render (10000). Defaults to 3000 when running locally.	—

In the Render Environment tab the value is typed **without** quotation marks. Saving an environment variable makes Render redeploy automatically.

3.4 Upstash Redis

- A free Redis database with a **REST API** (the server talks to it with plain HTTPS POST requests carrying a Redis command such as `["SET", "key", "value"]` and the bearer token). No extra npm package is needed and it works on any Node version.
- Free-plan limits as read on 24 Sep 2026 (re-verify): about 256 MB storage and 500,000 commands per month, data is persisted. The table is tiny; the server reads it once at start-up and writes only when the admin changes something, so usage is negligible. A live match will add roughly one write per finished hand.
- Keys used: `kalak3patti:table` (the tournament table, JSON). Planned: `kalak3patti:match` (snapshot of the live match).
- To inspect data: Upstash → the database → **Data Browser**. To rotate the token: Upstash → reset the password/token → paste the new token into Render → Render redeploys.
- If Upstash cannot be reached at start-up the API answers HTTP 503 ("Saved table temporarily unavailable") instead of pretending no table exists.

3.5 Deploy checklist (give this to the owner every time)

1. Upload the changed file(s) to GitHub at the exact path and press **Commit changes**. Note the short commit id GitHub shows (7 characters).
2. Render → the service → **Manual Deploy** → **Deploy latest commit**. Do not press it twice.
3. On the deploy page check **Source** = the new commit id. If it shows an older id, the upload did not reach *main*.
4. Wait for "**Your service is live**". A 12 MB page can make the deploy take a few minutes.
5. Open **Logs** → find the start-up lines: Table storage: Upstash Redis (survives restarts). then KALAK 3PATTI server listening on :10000. "local file" means the Upstash variables were not picked up.
6. Hard-refresh the game (Ctrl+Shift+R, or clear site data on the phone) so the old cached page is not shown.
7. Run the smoke test: admin login → create table → join from a second phone → see the countdown → kill the table.

Rollback: Render keeps earlier deploys (Events page) and can redeploy an older one; or revert/replace the file on GitHub and deploy again. Before any risky change, keep a copy of the last working KALAK_3PATTI.html and server.js.

3.6 Local test set-up an AI should recreate

- npm ci in a copy of the repo; run ADMIN_PASSWORD=test PORT=3111 node server.js.
- A 20-line fake Upstash (a tiny Node HTTP server answering GET / SET / DEL on a Map) lets tests run the real Upstash code path: set UPSTASH_REDIS_REST_URL=http://localhost:3222 and any token.
- Playwright (Python or Node) with **one browser context per simulated phone** (separate storage = separate device). Restart the server process in the test to prove persistence.
- For the live match: make every clock scalable (a timeScale option) so 10-minute windows can be tested in seconds.

4. Repository map and the "two versions of the game" problem

Path	What it is	Notes
server.js	Express server: static files, admin + join API, Upstash storage, brute-force guard.	Only dependency: express ^4.19.2. Will gain ws and a game engine module.
package.json / package-lock.json	Node dependencies and start script.	Any new npm package requires updating BOTH files (run npm install <pkg> and upload both).
public/KALAK_3PATTI.html	The whole game: HTML + CSS + about 4,800 lines of JavaScript, pictures and sounds embedded as base64.	The web root is public/ only; GET / serves this file.
gitignore	Meant to be .gitignore.	Rename to .gitignore (Phase 0) so node_modules and local files are never committed.

THE TWO-VERSIONS PROBLEM (read this before touching the game file)

The owner improves the game (new sounds, timer, animations...) in a **local master file** that also contains the old browser-only tournament code and an admin password typed into the page. That file must **never** be uploaded to GitHub. What goes to GitHub is the **server-connected version** produced from it by a conversion. On 25 Sep 2026 a local-style file replaced the server-connected one in commit 3d4dddd; the live site quietly stopped using the server for the table until it was fixed in commit 8877ae9.

The conversion (what makes a file safe to upload)

1. Delete the admin-password constant and the local password check; submitAdminPass becomes an async call to POST /api/admin/login. The password is then kept only in memory (adminPassword) and re-sent with each admin call.
2. Add one helper api(path, body) that always answers {ok, status, data} and never throws.

3. `showAdminView('create-table')` calls `loadTableFromServer()` (`/api/admin/table/get`). `TOURNAMENT_CONFIG` becomes the admin's in-memory copy only — nothing is written to `localStorage`.
4. `confirmCreateTable` → `/api/admin/table`; `updateStartTime` → `/api/admin/table/start`; `kickPlayer` → `/api/admin/table/kick`; `killTable` → `/api/admin/table/kill`.
5. `submitJoinPass` → `/api/join`; the "You're seated" screen and its countdown read the table info the server returned (`joinTableInfo`), and re-check the server every 10 seconds (`pollJoinInfo`).

Quick check that a file on GitHub is the right one

```
grep -c "calender"      public/KALAK_3PATTI.html # must be 0 (no admin password typed in the page)
grep -c "kalak_tournament_config" public/KALAK_3PATTI.html # must be 0 (no browser-only table storage)
grep -o "/api/[a-z]*)" public/KALAK_3PATTI.html | sort -u # must list admin/login, admin/table(/get,/kick,/kill,/start), join
```

Recommendation (Phase 0): stop maintaining two versions. Make the server-connected file the only master and let the owner ask an AI to edit that file directly. The live match (online mode) will change the client heavily anyway, so the local-only tournament code should be retired then.

5. The server today: API and data model (VERIFIED, server.js as on GitHub)

Endpoint (all POST unless noted)	Body	Result / errors
GET / and static files	—	Serves <code>public/KALAK_3PATTI.html</code> and anything else in <code>public/</code> .
<code>/api/admin/login</code>	<code>{password}</code>	200 <code>{ok:true}</code> · 401 "Incorrect password" · 429 too many wrong tries.
<code>/api/admin/table/get</code>	<code>{adminPassword}</code>	200 <code>{config}</code> (config is null when no table). The config includes every seat password — admin only.
<code>/api/admin/table (create)</code>	<code>{adminPassword, config}</code>	200 <code>{ok, config}</code> · 400 with a readable reason (needs 2–8 seats, duplicate seat passwords, observer password equals a seat password, bad date/time) · 409 a table is already online.
<code>/api/admin/table/kill</code>	<code>{adminPassword}</code>	200 <code>{ok:true}</code> ; deletes the table from Upstash.
<code>/api/admin/table/kick</code>	<code>{adminPassword, seat}</code>	200 <code>{ok, config}</code> · 400 computer seats cannot be kicked · 404 no table. Marks the seat <code>kicked:true</code> .
<code>/api/admin/table/start</code>	<code>{adminPassword, matchStartDate "YYYY-MM-DD" or "", matchStartTime "HH:MM[:SS]" or ""}</code>	200 <code>{ok, config}</code> · 400 invalid date/time · 404 no table.
<code>/api/join</code>	<code>{password}</code>	Seat: <code>{kind:"seat", seat, name, tableInfo}</code> . Observer: <code>{kind:"observer", tableInfo}</code> . Adds <code>{ended:true, winner}</code> when the table has a result. 401 wrong password · 403 <code>{kicked:true}</code> · 404 no tournament · 429 · 503 storage unavailable. Computer seats can never be joined.

Protection built in: passwords are compared in constant time; 10 *wrong* passwords per minute per IP (any password endpoint) → HTTP 429 (only failures count, so a room of players on one Wi-Fi is fine); the server trusts Render's proxy header for the real IP; the table is validated and normalised on the server (names ≤ 24 characters, passwords ≤ 40, numbers clamped); kicked and result are owned by the server and cannot be set by a client.

Table configuration (JSON stored under `kalak3patti:table`) — values shown are examples

```
{
  "numPlayers": 8,
  "players": [ { "seat": 1, "name": "Karan", "password": "<secret>", "type": "human", "kicked": false },
               { "seat": 2, "name": "Neha", "password": "<secret>", "type": "computer", "kicked": false }, ... ],
  "observerPassword": "<secret>",
  "chipsPerPlayer": 5000, "startingBoot": 10, "startingBlind": 10, "maxBlindCall": "Unlimited",
  "bootIncreaseMinutes": 5,
  "matchStartDate": "2026-09-26",           // "" when not set; entered/read as IST
  "matchStartTime": "19:30:00"              // "" when not set
  // PLANNED (not written today): "status": "waiting" | "live" | "ended",
  //                               "result": { "winner": "Neha", "winnerSeat": 2, "finalStack": 21000, "endedAt": 1790000000000 }
}
```

Seats vs chairs. The table config numbers seats 1...N (N = 2-8). The game screen always has **8 chairs** (0-7), laid out so the local player sits at the bottom. A table of N seats is spread around the 8 chairs like this (seat 1 → first chair in the list): 2 seats: 0,4 · 3: 0,3,5 · 4: 0,2,4,6 · 5: 0,2,3,5,6 · 6: 0,1,3,4,5,7 · 7: 0,1,2,3,5,6,7 · 8: 0-7. Empty chairs are hidden and skipped like a packed seat.

6. The game as implemented — the rulebook a server engine must reproduce

Source: the offline engine inside `public/KALAK_3PATTI.html` (VERIFIED by reading the code, 25 Sep 2026). A live server must behave identically, so the same hands produce the same results on the server as they do today in the browser.

6.1 Hand ranking (highest first)

Rank	Hand	How two hands of the same type compare
1	Trail (three of a kind)	Higher card rank wins (A high).
2	Pure sequence (same suit)	Highest card of the run; A-2-3 is the highest sequence , then A-K-Q, K-Q-J ... down to 4-3-2.
3	Sequence (mixed suits)	Same order as pure sequence.
4	Colour (same suit, not a run)	Compare the three cards from highest to lowest.
5	Pair	Higher pair, then the odd card.
6	High card	Compare the three cards from highest to lowest.

Ranks 2-14 (Ace = 14). Suits (clubs, diamonds, hearts, spades) have no rank, so exact ties can happen. **Tie on SHOW → the player who called SHOW loses. Tie on a sideshow → the player who asked loses.**

6.2 One hand, step by step

- The winner of the previous hand becomes dealer (first hand: a random seated player). If the boot went up, the new boot applies now — never in the middle of a hand.
- A seat that has stood up, is empty, or cannot afford the boot sits this hand out (shown "OUT"). Everyone else is "in".
- Boot collection** (~1.6 s): every seat that is in pays the boot into the pot; nobody can act yet.
- Deal:** three cards each, clockwise, starting with the seat to the dealer's left (about 110 ms between cards).
- Stake for the hand = **2 × boot**. The seat to the dealer's left acts first; play goes clockwise, skipping packed and sat-out seats.
- Each turn lasts **30 seconds**. If it runs out the seat is packed automatically. A seat that cannot afford the cost of its move is also packed automatically (no all-in, no short call).
- The hand ends when only one seat is left (that player wins; their cards stay hidden) or when someone calls SHOW (both hands are revealed).
- The winner collects the pot; a result screen shows for 7 seconds, then the next hand starts by itself.
- After the payout, if **one or fewer seats can afford the boot**, the match is over and the last one standing is the King.

6.3 Actions

Action	Allowed when	Cost	Effect
SEE	Any time you are in the hand and have not seen (even while a sideshow is pending). Not a turn.	free	You may look at your cards; from now on you pay the full stake.
BLIND	Your turn, cards unseen.	half the stake	Turn passes.
CALL	Your turn, cards seen.	the stake	Turn passes.
RAISE ×2	Your turn and you can afford it.	new stake if seen, half of it if blind	Stake doubles for everyone behind you. No cap today.
PACK	Your turn.	free	You are out of the hand.
SHOW	Your turn and exactly 2 seats are in.	the stake (half if blind)	Both hands compared; better hand takes the pot.
SIDESHOW	See 6.4.	free to ask	Private comparison with the previous seat; the lower hand packs.

6.4 Sideshow

- Needs 3 or more seats in the hand. The asker must have **seen** and be able to afford its normal move.
- The target is the nearest seat **before** the asker (counter-clockwise) that has not packed. That seat must have **seen**; if it is still blind there is **no sideshow at all** — the asker may not skip past it.
- One ask per turn, whatever the answer. The target has 15 seconds; **no answer = denied**. A denial changes nothing: the asker still has to make a normal move this turn.
- If accepted, both hands are compared at once. The lower hand packs (tie: the asker packs). If the asker lost, the turn passes on. If the asker won, it is still the asker's turn, but they may not ask again.
- The two players see both hands on a reveal screen (7 s, then it continues). Everyone else only sees a message. All other actions except SEE are frozen while a sideshow is pending.
- The turn clock keeps running through the whole sideshow; if it expires the sideshow is dropped and the asker is packed.

6.5 Boot escalation, stand up, game over

- **Boot** = $\min(\text{start boot} \times 2^{\text{level}}, \text{cap})$ where level = whole intervals of match time elapsed (default: doubles every 5 minutes; ₹10 → ₹640, the cap being 64 × the start boot). Match time is real clock time, not hand count. A new level is only *queued* and takes effect at the next deal.
- **Stand up** (menu): the seat gives up its chair — all its chips are **surrendered** (they leave the game), its hand is packed, it sits out every later hand, and the player keeps watching. It cannot sit again.
- **Game over**: after each payout, and also right after a stand-up between hands, if at most one seat can pay the boot. The match timer stops.

6.6 Timing constants in the newest game file

Name in the code	Value	Meaning
TURN_SECS	30 s	Time to act before auto-pack.
SIDESHOW_REQ_SECS	15 s	Time for the asked seat to answer; no answer = denied.
AUTO_CONTINUE_SECS	7 s	Next-hand countdown and the sideshow reveal auto-continue (older text in comments still says 5).
BOOT_ANIM_MS	1600 ms	Boot-collection beat before the deal.
DEAL_GAP_MS / DEAL_FLIGHT_MS	110 / 500 ms	Card dealing animation timing.
first turn delay	700–1400 ms	Pause after the deal before the first seat's clock starts.
bot think time	650–1550 ms (280–700 ms if no human is still in the hand)	Delay before a bot acts.

Name in the code	Value	Meaning
START_CHIP / BOOT_START / MAX_BOOT / BOOT_ESCALATE_MS	5000 / 10 / 640 / 5 min	Offline constants. The live engine must take start chips, start boot and interval from the table config.
NUM_SEATS	8	Chairs on screen.

6.7 Bots (Computer seats)

Each chair has a personality (aggression, bluffing, tightness, how soon it peeks). A bot works out its real chance of winning — exactly against one opponent (all 18,424 possible opponent hands are checked) or by simulation against several — compares it with the pot odds, then chooses pack / blind / call / raise / show / sideshow. It also decides whether to accept a sideshow. Computer seats on a live table will need this brain on the server. **The restricted bot for absent humans (section 8.4) is a different, much simpler mode and must not use this brain.**

6.8 Function index of the game page (to find things quickly)

Area	Names to look for
State and constants	G (the whole game state), NAMES, NUM_SEATS, LOCAL_SEAT (which chair is "me" — the screen rotates so "me" sits at the bottom), TURN_SECS.
Rules / engine	startRound, dealNewHand, advance, beginTurn, startTurnTimer, foldPlayer, packIfBroke, availableActions(idx) (pure: what a seat may do), performAction(idx,type), payIn, resolveRound, seatsInGame, standUp, endGameIfDecided, showKing, checkBootEscalation.
Sideshow	sideshowTarget, requestSideshow, respondSideshow, startSideshow, closeSideshow, abortSideshow.
Bots	botTurn, exactEquityVsOne, sampleEquityVsMany, estimateWinProbability, BOT_PERSONA.
Drawing	render, renderPlayerBigCards, updateActionBar, buildSeats, placeSeats, animateCollectBoots, animateDealCards, tossChip, potToSeat; sounds: playChipSound, playWinSound ...; match clock: startMatchClock.
Overlays (element ids)	winner-overlay, sideshow-overlay, ssreq-overlay, king-overlay, join-seated-overlay, match-ended-overlay, menu-panel.
Test-only feature	switchSeat / the "□ Seat" button lets one screen take over any seat. It must be hidden or disabled in live mode.
Tournament (server-connected)	api, submitAdminPass, loadTableFromServer, confirmCreateTable, refreshCreateTableState, updateStartTime, renderKickList, kickPlayer, killTable, submitJoinPass, showJoinSeated, pollJoinInfo, showMatchEnded, recordMatchResult (dormant).

7. What changed from the 24 September spec (v1 → v2)

Nothing in v1 was reversed. The changes below fill gaps that only became visible after the game file, the server and the deployment were reviewed.

#	Change	Why
1	Added precise definitions (8.1): entered the match, active session, presence, genuine action, timed-out round, bot-held seat, remaining players.	v1 used these words without defining them; an implementer could read them differently.
2	"Entered the match" now means a live authenticated connection at match start — not just having seen the waiting screen.	Today the waiting screen is a plain web request; only a live connection shows who is really present.
3	Added automatic start at the scheduled IST time plus a manual START MATCH button.	v1 assumed a start time but nothing in the system starts a match.
4	Restricted bot fully specified: always BLIND (never sees), then PACK; denies every sideshow request; cannot raise, show or ask.	v1 said "one minimum blind then fold" but left sideshow requests and unaffordable blinds open.
5	Genuine actions now also include raise, asking/answering a sideshow and standing up. An automatic pack (timer or unaffordable) is never a genuine action. A timeout during a sideshow counts.	Avoids resetting or missing the timeout streak by accident.

#	Change	Why
6	A player who is connected but idle for 3 rounds is treated exactly like a disconnected one.	The rule is about missed turns, not the network.
7	"Explicitly leaves the seat" defined as pressing EXIT THE TABLE or STAND UP. Closing the tab, refreshing or losing signal is not an explicit leave.	Refreshing must not unlock the seat for someone else.
8	Heartbeat numbers: server pings every 10 s; a device is gone after about 30 s without an answer. Each device has a session id, so a refresh or reconnect of the <i>same</i> device is never mistaken for a second person.	Makes the 30-second lock implementable and safe.
9	"Remaining players" for the chip split = seats still in the game (able to pay the current boot); standing, kicked, empty and eliminated seats get nothing. Distribution happens before the game-over check.	Prevents chips reviving an eliminated seat; fixes the order of events.
10	Admin Kick during a match = same as stand-up (chips surrendered, no split); it also works on a bot-held seat.	v1 split chips only for automatic kicks.
11	All deadlines (10-minute window, timeout streak, presence) are saved as absolute times so a server restart cannot reset or extend them.	Render can restart the service at any time.
12	Live observers are out of scope for the first live release; observers keep the waiting / match-ended screens.	The game screen is built around a seated player.
13	Match-end behaviour recorded (owner decision): no auto-exit, "MATCH HAS ENDED", late joiners see only the winner.	Already built in the client and in /api/join.
14	Round gap and reveal timing updated to the newest game file (7 s), boot collection and deal timings added.	The game file changed after v1.
15	A bot-held seat is shown to everyone (BOT badge and a countdown of the recovery window).	Fairness and clarity. PROPOSED.

8. Player Availability and Seat Protection Rules — v2 (final rule text)

The **server** is the only authority for seat ownership, presence, timeout counts, bot takeover, kick status and chip redistribution. The browser only displays state and sends the player's moves; it is never trusted to decide any of these.

8.1 Definitions

Term	Meaning
Human seat / Computer seat	Type chosen in Create Table. A Computer seat is a bot from the start and is not covered by these rules.
Match start	The moment the server starts the match: the scheduled date/time (IST) or the admin pressing START MATCH.
Entered the match	The player's device has an authenticated live connection (correct password) — at match start, or later while the seat is bot-held.
Active session	The one device (identified by its session id) that owns a seat right now through a live connection that keeps answering the server.
Presence	The device answers the server's heartbeat. The server pings every 10 s; after about 30 s without an answer the device counts as gone.
Genuine action	Blind, call, raise, pack pressed by the player, see, show, asking or answering a sideshow, stand up. Not genuine: a timer expiry, an automatic pack, logging in, heartbeat.
Timed-out round	A round in which the seat <i>received a turn</i> and its 30-second timer ran out, so the server packed it.
Bot-held seat	A human seat temporarily played by the restricted bot (8.4), with a recovery deadline.
Recovery window	10 minutes from the moment the bot takes the seat. The deadline is saved as an absolute time.

Term	Meaning
Remaining players	Seats that are still in the game after the kicked seat is removed: human, bot-held and computer seats that can pay the current boot. Standing, kicked, empty and eliminated seats are not included.

8.2 Player who never arrives

- At match start, every human seat with no authenticated live connection is taken over by the restricted bot. The 10-minute recovery window starts then.
- If the player enters the correct password before the window ends, the bot leaves and the player has the same seat with the chips that remain (8.6).
- If the window ends with no return, the bot stands up and the seat becomes **KICKED**; the password no longer works (8.7).

8.3 Player who disconnects or goes idle

- A connected player's turn lasts 30 seconds. A disconnect is **counted only** when that timer expires and the server packs the seat in **three consecutive rounds**. The bot then takes the seat **from the next round** and the 10-minute window starts.
- A round in which the seat never receives a turn (the hand ended before reaching it) neither increases nor resets the count. An automatic pack because the seat cannot afford its move is not a timeout.
- Any genuine action resets the count to zero.
- A player who is connected but idle for three rounds is treated exactly the same. Being offline alone never triggers the bot — only missed turns do. (Each timed-out turn can hold the table for up to 30 s; the owner accepts this.)

8.4 The restricted bot (per hand)

1. It pays the boot like every seat (if it cannot afford the boot the seat sits the hand out).
2. On its **first turn** in the hand it plays **BLIND at the minimum** (half the current stake). It never looks at the cards. If it cannot afford that blind, the normal automatic pack applies.
3. On its **next turn** in the same hand it **PACKS**.
4. It never raises, calls, shows or asks for a sideshow. A sideshow request addressed to it is **denied at once**.
5. It acts after a short natural delay (about 0.7–1.5 s) instead of waiting for the 30-second clock, and it does not count timeouts.
6. If everyone else packs before its second turn it simply wins the pot like any seat. Its counters start again every hand.

8.5 Active-session seat protection

- When a player is inside the match the server marks the seat as actively occupied by that device. A second login with the same password from a **different device** is **rejected** ("This seat is already in use on another device"). It cannot replace, disconnect or take control of the active player.
- The same device (same session id) may reconnect at any time — refresh, brief network drop, a second tab of the same browser — and simply replaces its own old connection.
- The lock is released when the original device has shown no presence for about 30 seconds, or when the player explicitly leaves (EXIT THE TABLE or STAND UP).
- Releasing the lock never triggers the bot by itself. Only the three-timed-out-rounds rule (8.3) or a never-arrived seat (8.2) does.

8.6 Returning during bot control, and device replacement

- A player reclaims a bot-held seat by entering the correct password during the window (through the normal join screen). The bot leaves at the next safe point: immediately if it is not that seat's turn, otherwise as soon as the bot has finished the move it is making. The timeout count resets to zero, the deadline is cleared and the chips stay as they are.
- After the window ends and the seat is KICKED, the password is refused for good.

- **Device replacement:** if a phone fails, its session stays protected for about 30 seconds (a second login is rejected). After that another device may log in, subject to the normal rules above — if the seat is bot-held this is simply a return.

8.7 Automatic kick and chip distribution

Applies **only** when a seat is kicked automatically at the end of its 10-minute window. Steps, in this order:

1. The bot stands up: if the seat has a hand running, that hand is packed (chips already in the pot stay in the pot). The seat is marked **KICKED** in the table (saved to Upstash), so `/api/join` refuses the password.
2. Take the seat's remaining chips **C**. Count the remaining players **N** (defined in 8.1).
3. $\text{share} = C \div N$, rounded **down to a multiple of 10**. Every remaining player receives the share at once.
4. Remainder **R** = $C - \text{share} \times N$ goes into the pot: the running hand's pot, or — if no hand is running — a carry pot that is added to the next hand's pot right after its boots are collected. That hand's winner receives it.
5. Then the game-over check runs (if at most one seat can pay the boot, the match ends). Everyone sees a message: the seat, C, the share and R.

Example	Value
Kicked seat's chips (C)	4,730
Remaining players (N)	7
Share per player	670 ($4,730 \div 7 = 675.7 \rightarrow 670$)
Distributed in total	4,690
Remainder to the pot (R)	40

8.8 Voluntary stand-up and admin kick

- **Voluntary stand-up:** unchanged — chips are surrendered (removed from play), no split.
- **Admin KICK during a match** (cheating, etc.): same as stand-up — surrender, no split — plus the password becomes invalid. It works on a bot-held seat too.
- The admin panel should show, per seat: connected / offline / bot-held (time left) and the timeout count. Kick and Kill keep their confirmation taps.

8.9 What the server must own, and how it survives a restart

1. Password entry and seat assignment.
2. Active sessions and device ownership.
3. Presence signals and the ~30-second lock.
4. The count of consecutive timed-out rounds per seat.
5. Bot takeover state and the recovery deadline (absolute time).
6. Return of a player and removal of the bot.
7. Permanent KICKED status.
8. Chip redistribution and the carry pot.

Persistence (PROPOSED): per seat the server saves `botHeld`, `botSince`, `recoveryDeadline`, `timeoutStreak` in the match snapshot (Upstash key `kalak3patti:match`) whenever one of them changes and at the end of every hand. After a restart the match resumes at the next deal (about 8 seconds after start-up), players reconnect automatically, and deadlines keep counting on the wall clock — time the server was down counts against the window. (The alternative, pausing deadlines during downtime, is possible but needs the server to know it was down; wall-clock is simpler and predictable.)

8.10 Timing summary

Situation	Rule
Player never arrives	Restricted bot at match start; 10-minute window starts.
Player returns during the window	Bot leaves at the next safe point; chips as they are; counters reset.

Situation	Rule
Nobody returns for 10 minutes	Bot stands up; seat KICKED; password invalid; chips shared (8.7).
Active session	A second login from another device is rejected.
Original device silent ~30 s	Login lock released (no bot yet).
3 consecutive timed-out rounds	Bot takes over from the next round; 10-minute window starts.
Any genuine action	Timeout count back to zero.
Voluntary stand-up / admin kick	Chips surrendered, no split.
Server restart	Match resumes at the next deal; deadlines are absolute times.

9. Roadmap: from today to a live tournament

Order matters: every phase depends on the one before it. Do not start a phase until the previous one passes its acceptance tests and is deployed and confirmed by the owner. Sizes are relative effort: S = small, M = medium, L = large.

Phase	Goal	Size	Status (25 Sep 2026)
0	Baseline, housekeeping, one master file, test kit	S	Partly done (deploy works; the rest open)
1	Server-side game engine (rules only, no network)	L	Not started
2	Real-time connection (WebSocket), sessions, heartbeat	M	Not started
3	Game page "online mode" (play through the server)	L	Not started
4	Match lifecycle: start, results, restart safety	M	Not started
5	Availability and seat protection (section 8)	L	Not started
6	Hardening: security, fairness, monitoring, Render plan	M	Not started
7	Rehearsal with real phones and launch	S	Not started

Phases 1 and 2 can be developed side by side. The first time real players can sit at one table together is the end of Phase 4; the absent-player rules arrive in Phase 5.

Phase 0 — Baseline and housekeeping (S)

- **Confirm the deployed state** with the smoke test in 3.5 (commit 8877ae9 or newer). If a table from earlier tests is still online, kill it.
- Rename gitignore to .gitignore with: node_modules/, tournament-config.json, match-state.json. (GitHub's web uploader cannot create dot-files easily — use "Create new file" and type .gitignore as the name.)
- **One master file:** make the server-connected game page the only master (section 4). Retire the local-only tournament code and the typed-in admin password from any copy the owner keeps.
- Make deploys checkable: log the deployed commit at start-up (Render provides the environment variable RENDER_GIT_COMMIT — verify) and show it in the admin panel footer, so "did my upload go live?" is answered on screen.
- Put this PDF in the repository (for example docs/) so any AI that opens the repo finds it.
- Create the test kit (3.6): fake Upstash + multi-browser Playwright, kept in a tests/ folder. Not needed by Render, harmless in the repo.

Acceptance: deploy shows the commit id; .gitignore works; the three greps in section 4 pass on the deployed file.

Phase 1 — Server-side game engine (L)

- New module game.js (plain CommonJS, no browser code) implementing section 6 exactly: 8 chairs, hand evaluation, boot collection / deal / turn flow, all actions, sideshow, show, auto-pack, boot escalation, stand up, game over.

- Read start chips, start boot, boot interval (and cap = 64 × start boot) from the table config. Decide what to do with startingBlind and maxBlindCall (open decision 6).
- **Fair shuffle:** use Node's crypto.randomInt (the browser game uses Math.random, which is fine offline but not for a real tournament).
- Make time injectable (timeScale) and the random source injectable so tests are repeatable.
- The engine emits **events** (deal, boot collection, bet, see, fold, show, turn, sideshow ask / deny / reveal / close, round end, game over, messages) and can build a **per-viewer view** that never contains hidden cards (Appendix B).
- Port the bot brain (6.7) for Computer seats; keep the restricted bot (8.4) as a separate tiny mode for Phase 5.
- Snapshot / restore functions for use between hands (Phase 4).

Acceptance: (a) unit tests for every hand type, both tie rules, A-2-3, sideshow branches, unaffordable auto-pack, boot escalation timing, stand up, game over; (b) a simulation of at least 5,000 hands with 8 bots finishes without a stuck turn, without negative chips, and total chips + pot are conserved except for surrendered stand-up chips; (c) the server's hand evaluation gives identical results to the browser's for 100,000 random pairs.

Phase 2 — Real-time connection (M)

- Add the ws npm package (npm install ws; upload the new package.json and package-lock.json). Attach a WebSocket server at /ws to the same HTTP server Express uses. Render supports WebSockets on web services (verify in Render's docs when starting).
- Authentication: first message {t:"auth", password, sid}. Reuse the existing password logic and the 10-wrong-per-minute guard. Check the Origin header, cap message size (a few KB) and message rate.
- Session rules from 8.5: same sid replaces its own connection; a different sid while another is alive is rejected. Heartbeat: ping every 10 s, terminate after ~30 s silence. Explicit leave message releases the lock.
- Client-side auto-reconnect with the same session id and a visible "reconnecting..." state.
- Server → client messages carry only what that player may see (Appendix A/B). Lobby messages replace the 10-second polling on the waiting screen (start-time change, kicked, table killed, match live, match ended).

Acceptance: two browser contexts, same password → second refused; refresh → accepted; cutting the first connection → second device accepted after ~30 s; wrong-password guard still works; a spectator of the raw WebSocket frames never sees another player's cards.

Phase 3 — Game page "online mode" (L)

- Leave offline mode untouched. Add an **online mode** entered from the waiting screen when the match goes live.
- Mirror the server view into the existing G and NAMES, set LOCAL_SEAT to the player's chair, and reuse render(), availableActions, the overlays, sounds and animations. Do not run startRound / advance / botTurn / resolveRound / timers locally.
- Buttons send messages (action, ss_answer, stand, leave, avatar) instead of calling performAction. Sideshow accept/deny, Stand Up and Exit Table are re-wired the same way.
- Animations and sounds are driven by server **events** (boot collection, deal, chip toss, fold, show, round end, king). Turn clocks are drawn from server timestamps corrected by the measured clock offset; the client never folds anyone.
- Hide empty chairs (seats not in use), the test-only "Seat" switch, and anything that lets a screen control another seat. Table theme stays a local, per-player choice.
- Messages arrive with tokens such as @3 that each phone replaces with "You" or the player's name.
- Keep the match timer, boot-collection pause and 7-second round gap exactly as in the offline game (server-scheduled).

Acceptance: three real or simulated phones plus computer seats play complete matches end to end; every rule in section 6 behaves the same as offline; no rule decision is made in the browser (search the online path for direct calls to performAction); screenshots match the offline look.

Phase 4 — Match lifecycle (M)

- Add status ("waiting" → "live" → "ended") and result to the table config. Add admin endpoints and buttons: **START MATCH** (with a confirmation tap) and status display (connected seats, round, boot level).
- **Automatic start** at `matchStartDate + matchStartTime` interpreted as IST. Decide what happens if the server was asleep at that moment (open decision 1).
- At start, seat connected humans, computer seats and — from Phase 5 — bot-held seats for absent humans. Until Phase 5 exists, unconnected human seats simply sit out (say so on the admin panel).
- On the last hand: write result to the table (winner, seat, final stack, time) — the client's match-ended screens and `/api/join` already understand it. Killing the table clears everything.
- Save the match snapshot to Upstash at the end of every hand; on start-up resume from it (next deal after ~8 s). Scan for the failure case: Upstash unreachable during a match (keep playing in memory, warn the admin, retry).

Acceptance: a full match from START MATCH to the King screen; a late joiner sees only the match-ended screen; killing the table mid-match closes every phone cleanly; restarting the server mid-hand resumes at the next deal with correct chips.

Phase 5 — Availability and seat protection (L)

- Build in this order, testing each: (a) session lock and heartbeat (from Phase 2) → (b) timeout streak counting → (c) the restricted bot → (d) the 10-minute window with absolute deadlines → (e) return by password → (f) automatic kick with the chip split and carry pot → (g) admin panel visibility (per-seat state, time left, timeout count) and the BOT badge on the table.
- Use the `timeScale` option so 10-minute windows and 30-second locks can be tested in seconds; then run one real-time test for each.
- Add the new fields to the match snapshot (8.9).

Acceptance: every scenario T1-T20 in section 10 passes.

Phase 6 — Hardening (M)

- Security review: no secrets in the page or logs; every admin action re-checks the password; WebSocket origin check; input validation on every message; rate limits; error messages that do not leak seat lists.
- **Hidden information audit:** inspect real WebSocket frames for each role (player, kicked player, observer) and prove no foreign cards are ever sent.
- **Hand history:** log every hand (who was dealt what, every action, result) so a dispute can be settled later; store in a capped Upstash list or a log service.
- **Render plan:** the free plan can restart or sleep at any time and cold-starts slowly. For real events consider a paid instance (always on). Decide before launch.
- Load and soak tests: 8 simulated phones for hours; forced restarts; Upstash outage; slow networks.

Phase 7 — Rehearsal and launch (S)

- Dress rehearsal with 4-8 real phones on real mobile data: play a full match; kill Wi-Fi mid-hand; refresh; log in from a second device; arrive late; restart the server mid-hand.
- Launch-day checklist for the owner: server deployed and commit verified; `ADMIN_PASSWORD` and both Upstash variables set; create the table; set the start date/time (IST); send each player their own password privately; check the `kalak3patti:table` key in Upstash; keep the admin panel open during the match.
- After the match: read the result, kill the table, keep the logs. Write down lessons and update this document.

Later (not part of the first live release)

Live spectators for the observer password, several tables at once, chat, wallet/payments, rankings, replays.

10. Scenario test matrix (acceptance tests for Phases 2-5)

ID	Scenario	Expected result
T1	A human seat never connects.	At start the restricted bot holds it; BOT badge + 10-minute countdown shown.
T2	That player enters the password at minute 4.	Bot leaves at the next safe point; chips as they were; countdown gone; streak = 0.
T3	Same player arrives at minute 11.	Seat is KICKED; /api/join answers 403 kicked; chips were shared.
T4	Player answers nothing for 3 consecutive rounds (each turn timed out).	Bot takes over from the next round; window starts.
T5	Two timeouts, one CALL, two more timeouts.	No takeover (the CALL reset the streak).
T6	Rounds where the seat never got a turn between timeouts.	Streak unchanged by those rounds.
T7	Second device logs in with the same password while the first is active.	Rejected: "already in use"; first device undisturbed.
T8	Same device refreshes / reconnects / opens a second tab.	Accepted; old connection replaced.
T9	First phone dies; second phone tries at 10 s, then after 35 s.	Rejected at 10 s; accepted after ~30 s (as a normal or bot-held return).
T10	Chip split: C = 4,730, N = 7.	Each +670; 40 to the pot; message shown.
T11	Kick expires while a hand runs / between hands.	Remainder goes into that hand's pot / into the next hand's pot; winner receives it.
T12	Player presses Stand Up.	Chips surrendered, no split, seat watches.
T13	Admin kicks a player (and a bot-held seat) mid-match.	Surrender, no split, password invalid, that phone told it was removed.
T14	Server restarts mid-hand.	Resume at next deal; chips, bot-held seats and deadlines intact.
T15	Restricted bot over several hands.	Boot, one minimum blind, then pack; never sees; sideshow requests denied.
T16	Bot holds the seat when its player returns during that seat's turn.	Bot finishes its move, then control passes; no double action.
T17	Kicked password tried on join and on the live connection.	Refused both ways.
T18	Match ends.	Result stored; King screen with EXIT button; a later joiner sees only the match-ended screen.
T19	Admin kills the table during a match.	Every phone is closed with a message; storage cleared.
T20	Read WebSocket frames as player A.	Never contains B's cards (except a real showdown or A's own sideshow).

11. Traps to avoid and decisions still open

11.1 Traps (things that have misled or will mislead)

- Uploading the **local master file** to GitHub (typed-in admin password, browser-only table). Always run the greps in section 4 on the file before it goes to GitHub.
- Trusting a runtime file for persistence on Render (it is wiped). Use Upstash.
- Assuming a change is live after uploading to GitHub. It is live only after the Render deploy shows the new Source commit and "live".
- Adding an npm package but uploading only package.json. Upload package-lock.json too.
- Confusing config seats (1...N) with chairs (0...7).
- Putting a secret in a file or asking the owner to paste one. Environment values are typed into Render without quotation marks.
- Letting the browser decide rules or presence. The server decides; the browser draws.

- Sending every player's cards to every phone "for convenience". Never: send only what that viewer may see.
- Changing the offline game while building online mode. Both must work.
- Relying on the GitHub API from a script without authentication (rate-limited); use the atom feed and raw URLs, or ask the owner.
- Shrinking the game file (the owner decided not to).
- Testing only with one browser. Use separate contexts (separate storage) to simulate different phones.

11.2 Open decisions (recommended default in bold; the owner may change any of them)

#	Decision	Recommended default
1	Automatic start when the server was asleep or down at the scheduled time.	Start it as soon as the server is up if the time has passed by less than 15 minutes; otherwise wait for the admin.
2	Do deadlines keep counting while the server is down?	Yes (wall clock).
3	Sideshow requests to a bot-held seat.	Denied at once.
4	Live spectating for observers.	Not in the first release.
5	Render plan for real tournaments.	Paid (always on) for anything with real players.
6	Settings startingBlind and maxBlindCall (stored but unused).	Remove from the form or implement them in Phase 1; do not leave them as decoration.
7	Recipients of the chip split include players who cannot pay the boot?	No — only seats still able to pay the boot.
8	Show a BOT badge and countdown to everyone.	Yes.
9	Chips surrendered by stand-up / admin kick.	Removed from play (as today).

Appendix A — WebSocket protocol (DRAFT design, not implemented)

JSON messages, one per frame. t is the type. Client → server:

Type	Fields	Meaning
auth	password, sid	First message. sid = random per-device id kept in the browser.
action	a: pack see blind chaal raise show sideshow	A move. The single SHOW button is turned into a sideshow request by the server when that is what is allowed.
ss_answer	accept: true/false	Answer to a sideshow request addressed to this seat.
stand	—	Stand up (surrender chips).
leave	—	Explicit leave: releases the session lock.
avatar	idx: 0–19	Change avatar.

Server → client:

Type	Fields	Meaning
authed	kind (seat observer), seat, name, tableInfo	Password accepted.
lobby	status, tableInfo, inMatch, winner?	Waiting-screen updates: start time changed, match live, match ended.
update	state, events[]	A full per-viewer state plus what happened (for animations, sounds, messages).
denied	text	Why a move was refused.
error	code (in_use bad_password no_table rate), error	Login refused; the socket then closes.

Type	Fields	Meaning
kicked	—	This seat was removed by the admin (or automatically after the window).
table_killed	—	Admin killed the table.
replaced	—	The same device opened a newer connection.

Event kinds inside `update.events`: `deal`, `collect_boot`, `bet {seat, amount}`, `see {seat}`, `fold {seat}`, `show {seat}`, `turn {seat}`, `msg {text with @n tokens}`, `ss_ask`, `ss_deny`, `ss_reveal` (the two involved players also get both hands), `ss_close`, `round_end {winnerIdx, pot, showdown, label, winnerHand?, others?, gameOver, kingIdx}`, `game_over`.

Appendix B — What one player's view of the table may contain

Field	Notes
<code>you</code> , <code>now</code> , <code>names[8]</code>	Own chair, server time (for clock offset), the 8 chair names ("" for empty chairs).
<code>players[8]</code>	<code>name</code> , <code>chips</code> , <code>folded</code> , <code>seen</code> , <code>standing</code> , <code>empty</code> , <code>avatarIdx</code> , <code>bet</code> , <code>lastBet</code> , <code>handLen</code> , and hand only for : the viewer's own cards once they pressed SEE, or every unfolded hand at a real showdown.
<code>pot</code> , <code>currentBet</code> , <code>currentBoot</code> , <code>round</code>	Public numbers.
<code>currentIdx</code> , <code>dealerIdx</code> , <code>turnEndsAt</code>	Whose turn and when the clock ends (server time). -1 during boot collection.
<code>roundOver</code> , <code>showdown</code> , <code>gameOver</code> , <code>kingIdx</code>	Hand / match state.
<code>ssAskedSeat</code> , <code>ssAskDenied</code> , <code>sideshow</code>	Sideshow state (phase ask / reveal). Hands are never inside this object.
<code>later</code> (Phase 5)	<code>botHeld</code> and <code>recoveryDeadline</code> per seat, so everyone can see a BOT badge and its countdown.

Appendix C — Glossary

Word	Meaning
Boot	Entry chips every seat in a hand pays into the pot; doubles over time.
Stake / current bet	2 × boot at the start of a hand; a blind player pays half of it, a seen player pays all of it; RAISE ×2 doubles it.
Blind / seen	A blind player has not looked at their cards; "seen" means they pressed SEE.
Chaal / call	Pay the stake to stay in (seen players).
Pack	Fold.
Show	Compare the last two hands.
Sideshow	Ask the previous seat for a private comparison.
King	The last seat able to pay the boot: the winner of the match.
IST	Indian Standard Time, UTC+05:30. All dates and times entered by the admin are IST.
sid	A random id created once per device/browser; identifies "the same device" across refreshes.
Upstash / Redis	A small online database used to keep the table when Render restarts.

Appendix D — Quick reference

Item	Where
Code	https://github.com/walling844-klk/kalak-3patti (main)
Commit list (no login needed)	https://github.com/walling844-klk/kalak-3patti/commits/main.atom
Raw files	<a href="https://raw.githubusercontent.com/walling844-klk/kalak-3patti/main/<path>">https://raw.githubusercontent.com/walling844-klk/kalak-3patti/main/<path>

Item	Where
Live game	https://kalak-3patti.onrender.com
Render	dashboard.render.com → the service → Events / Logs / Environment / Manual Deploy
Upstash	console.upstash.com → the Redis database → Data Browser / REST API
Start-up log lines that prove a healthy deploy	"Table storage: Upstash Redis (survives restarts)." then "KALAK 3PATTI server listening on :10000"
Storage key	kalak3patti:table (planned: kalak3patti:match)

End of document. If anything here disagrees with the code in the repository, the code wins — then update this document.